



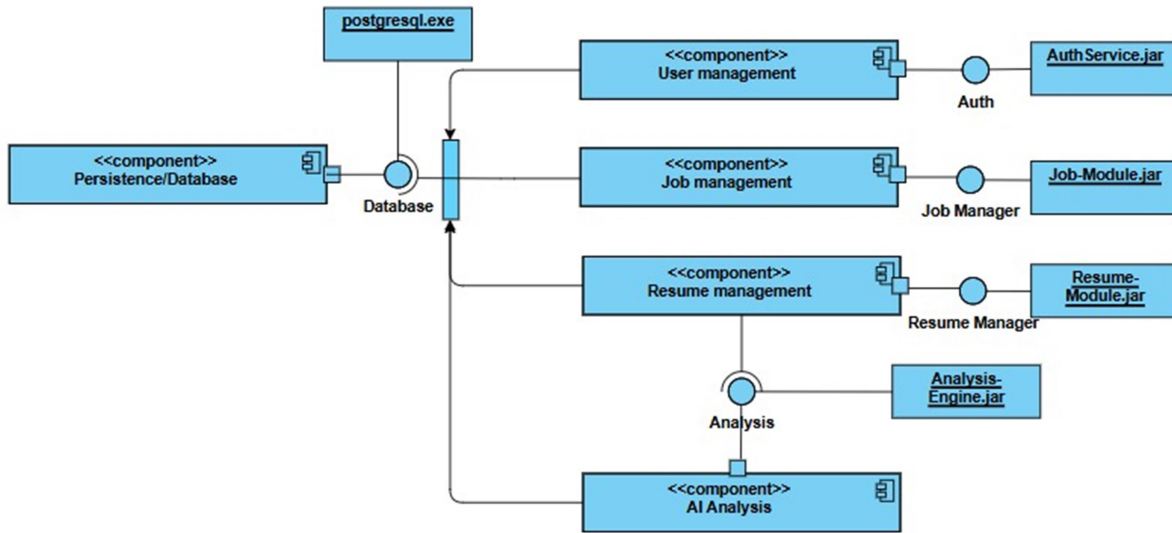
Assignment 3

Semester Project Title: AI Resume Analyzer

Student Details

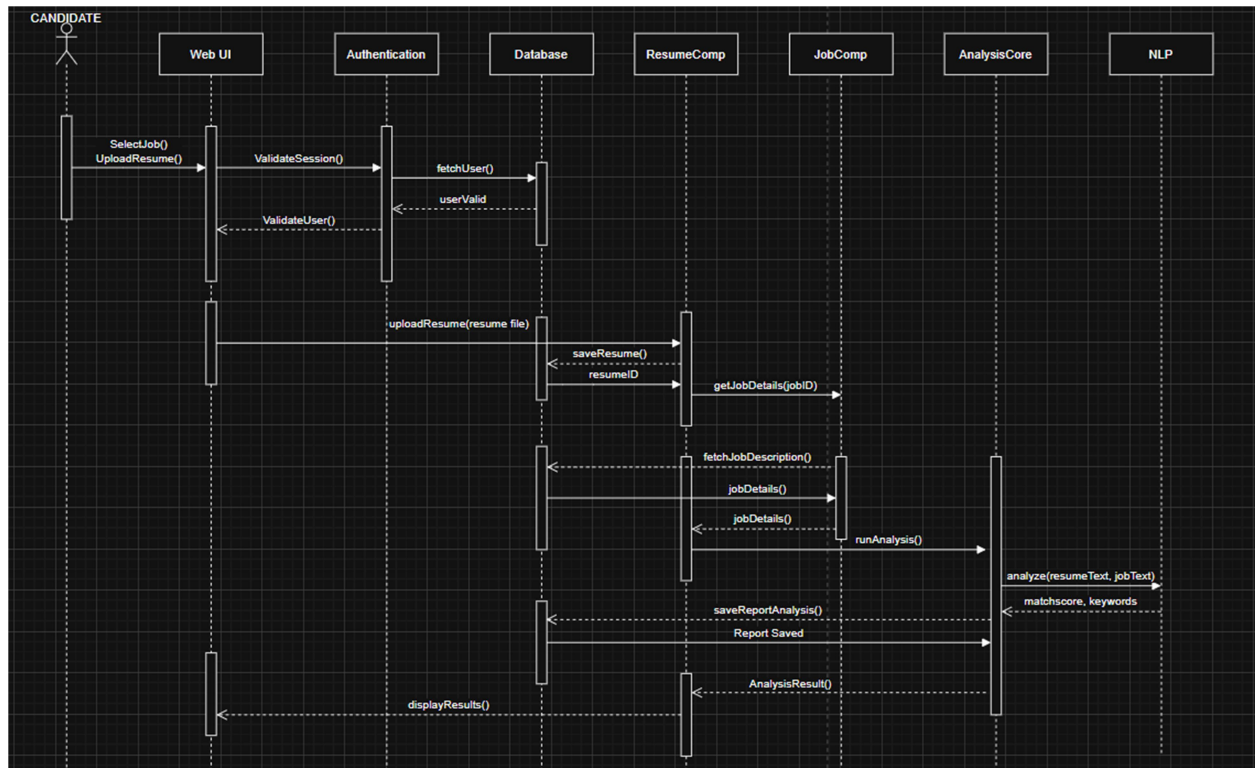
	Student Name	Student Reg #	Student Degree
Student-1	Aazeb Ali Malik	2023003	BSCSW
Student-2	Abdullah Ihsan	2023039	BSCSW
Student-3	Syed Faseeh	2023689	BSCSW

Section 1: Component Diagram



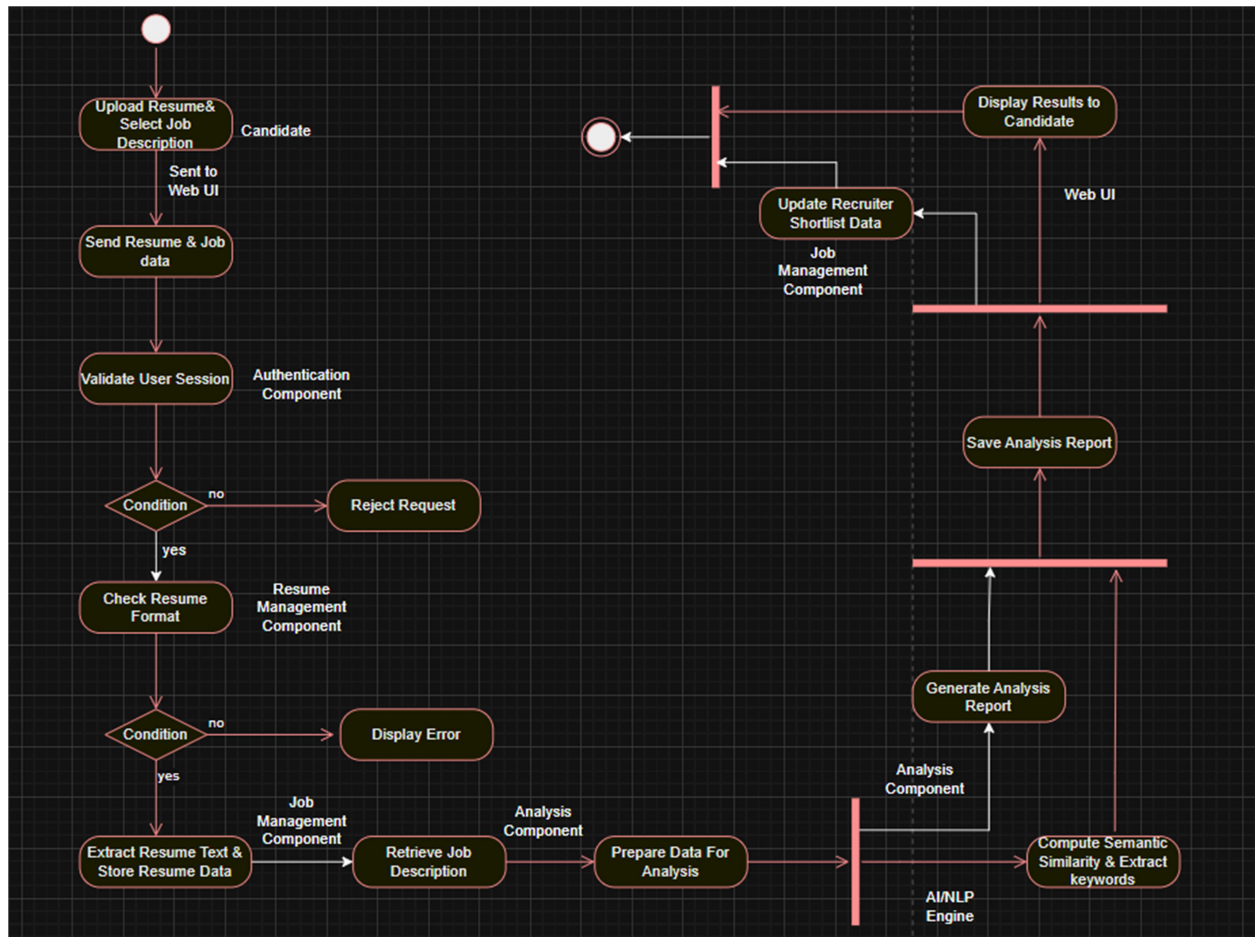
The diagram visualizes the system as five modular black-box components that interact exclusively through well-defined interfaces, ensuring a clear separation of concerns as outlined in the architecture. The User Management Component acts as the system's security gatekeeper by providing the **Auth** interface to the frontend for login and registration while relying on the Persistence Component to verify credentials and store user data. The Job Management Component supports recruiter operations by exposing the **JobManager** interface for posting, editing, and searching job descriptions and it connects to the Persistence Component to store job details and extracted keywords. The Resume Management Component manages candidate data by providing the **ResumeManager** interface for uploading and retrieving resumes and depends on the AI Analysis Component to trigger resume-job analysis when required. The AI Analysis Component serves as the core processing engine, exposing the **Analysis** interface to handle comparison requests, internally utilizing the **NLPModel** (via the Factory pattern) to compute semantic similarity, and persisting generated **AnalysisReports** through the Persistence Component. Finally, the Persistence Component forms the foundation of the system by encapsulating the PostgreSQL database and repository logic and exposing a **Database** interface that all other components use to store and retrieve their respective data objects, including users, resumes, jobs, and analysis reports.

Section 2: Sequence Diagram



This sequence diagram models the complete lifecycle of the resume upload and AI-based analysis use case. The interaction begins when a candidate uploads a resume and selects a job through the Web UI. The system first validates the user session via the Authentication component. The Resume & Analysis Orchestration component then stores the resume and retrieves job details from the Job Management component. The analysis request is delegated to the AI/NLP Core, which further invokes the NLP Model Implementation to compute semantic similarity and keyword matching. The generated analysis report is persisted in the database and finally returned to the Web UI for display. Activation bars clearly show processing responsibility, while dependencies between components demonstrate how the system fulfills the use case through coordinated, loosely coupled interactions.

Section 3: Activity Diagram



This activity diagram models the complete AI-based resume analysis process, starting from a candidate uploading a resume to receiving actionable feedback. The flow includes multiple decision points, such as session validation and resume format verification, ensuring system security and data correctness. Swimlanes clearly assign responsibilities to different components, including the Web UI, Authentication, Resume Management, Job Management, and AI/NLP Engine. Parallel activities are represented using fork and join nodes, allowing the system to simultaneously perform AI analysis and report generation, as well as update recruiter data while displaying results to the candidate. An activity diagram is well-suited for this process because it clearly visualizes control flow, decisions, and concurrency across system components, making complex business logic easy to understand and validate.

Section 4: Architectural Style and Rationale

Layered (n-tier) Architecture

The AI Resume Analyzer is designed using a Layered (n-tier) Architecture, consisting of three main layers: Presentation Layer, Application/Domain (Business Logic) Layer, and Persistence (Data Access) Layer.

1. Why does this architectural style fit the project's specific needs?

This architectural style fits the AI Resume Analyzer because the system combines multiple distinct concerns: user interaction through a web interface, complex AI-based resume analysis, and persistent storage of resumes and job data. The Presentation Layer handles all user-facing operations such as resume uploads, job postings, and result visualization. The Application/Domain Layer contains the core logic, including NLP-based resume–job comparison, match score calculation, and feedback generation. The Persistence Layer manages secure storage and retrieval of users, resumes, job descriptions, and analysis reports in a PostgreSQL database. By separating these responsibilities, the system remains organized and easier to extend, especially as AI models or features evolve.

2. How does this style help achieve the non-functional requirements?

The Layered Architecture directly supports key non-functional requirements such as maintainability, scalability, security, and performance. Separation of concerns allows each layer to be developed, tested, and maintained independently, making the system easier to update and debug. Scalability is improved because resource-intensive AI processing in the Application Layer can be optimized or scaled without affecting the UI or database layers. Security is enhanced by centralizing authentication, authorization, and data validation logic within the business layer, while the Persistence Layer enforces data integrity and encryption. This structure also improves reliability and availability by isolating failures within a single layer.

3. How does this architectural style relate to the design patterns chosen in Assignment 2?

The Layered Architecture integrates seamlessly with the design patterns defined in Assignment 2. In the Persistence Layer, the Repository pattern (UserRepository, ReportRepository) abstracts database access and ensures loose coupling between data storage and business logic. In the Application Layer, the Factory Method and Strategy patterns are applied through the NLPModel interface, allowing different AI and NLP algorithms to be swapped or extended without modifying the AnalysisService. The Adapter pattern

enables integration with third-party resume parsing libraries, while the Observer pattern supports real-time notification of analysis results to user interfaces. These patterns reinforce the layered structure and ensure extensibility, flexibility, and adherence to SOLID principles.